



# **POLITECNICO DI BARI**

DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELL'INFORMAZIONE (DEI)  
CORSO DI LAUREA MAGISTRALE IN INGEGNERIA DELL'AUTOMAZIONE

---

Final project

## **Digital Programmable Systems**

**"Car Parking System in VHDL"**

**Professor:**

Dr. Eng. Martino De Carlo

**Students:**

Rocco Falcone  
Savino Francesco  
Savino Tommaso

**ACADEMIC YEAR 2022 – 2023**

## **Index**

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>Abstract .....</b>                              | <b>3</b>  |
| <b>1. Introduction .....</b>                       | <b>3</b>  |
| 1.1. FPGA: DE10-Lite board .....                   | 4         |
| <b>2. Design and implementation .....</b>          | <b>6</b>  |
| 2.1 Finite State Machine.....                      | 6         |
| 2.2 Breadboard implementation.....                 | 13        |
| 2.3 Wrong Password Alert.....                      | 15        |
| 2.3.1 Buzzer Implementation .....                  | 15        |
| 2.4 HC-SR04 ultrasonic sensor implementation ..... | 18        |
| 2.5 The Parking Gate.....                          | 22        |
| 2.5.1 Servo Motor Working Principle .....          | 22        |
| 2.5.2 Servo Motor Implementation.....              | 23        |
| 2.5.3 FPGA Connection .....                        | 26        |
| <b>3. Conclusions .....</b>                        | <b>28</b> |
| <b>4. References .....</b>                         | <b>29</b> |

## Abstract

This paper aims to illustrate the realization of a VHDL based system of a car parking lot. The system has been designed for a final assessment project of Digital Programmable Systems course. This document contains concept, design, challenges, results, discussion, and conclusion.

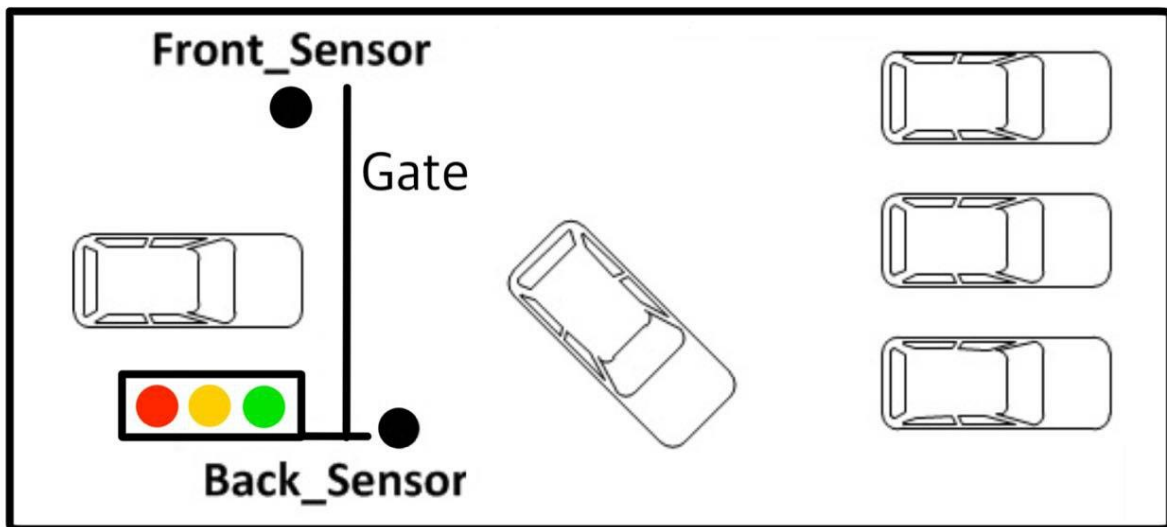
## 1. Introduction

The car parking system is developed using VHDL in Intel Quartus Prime Lite Edition software. In this project, the Altera DE10-Lite board was used with the following additional components:

- a breadboard,
- two HSR-04 ultrasonic sensors (front and back sensor),
- a SG-90 servo motor (Micro Servo 9g),
- a passive buzzer,
- three LEDs (red, yellow, and green) with associated 220 ohm resistors.

The parking was implemented through a finite state machine (Mealy machine) with 4 states that are signalled in the 7-segment display on the board.

A representation of the parking lot is shown in the following figure:



*Figure 1: representation of the parking lot*

## 1.1. FPGA: DE10-Lite board

Field Programmable Gate Arrays (FPGAs) are semiconductor devices that are based around a matrix of configurable logic blocks (CLBs) connected via programmable interconnects.

As suggested by the name of the semiconductor technology, advantages of FPGA are known for their ability to be programmed when implemented in the field as opposed to other kinds of semiconductor chips which are largely rigid in their design and execution.

Due to their programmable nature, FPGAs are an ideal fit for many different markets. The FPGAs are used in quite impressive range of different applications, such as [1]:

- **Aerospace & Defence** – Secure communication, waveform generation, ATM, control systems, radar, guidance, and control.
- **Automotive** - Driver assistance systems, comfort, convenience, and in-vehicle entertainment.
- **Communications Broadcast** – Wired communications, wireless communications (cellular connectivity, base stations, wireless LAN), transport and networking solutions.
- **Consumer Electronics** - Converged handsets, digital flat panel displays, information appliances, home networking, residential set top boxes, office automation (couplers, printers).
- **Data Center** - High-bandwidth, low-latency servers, networking, storage applications.
- **Industrial** - Industrial imaging and surveillance, industrial automation, and medical imaging equipment.
- **Medical** - For diagnostic, monitoring, and therapy applications.
- **Security** - Access control to surveillance and safety systems.

In this project, the Altera DE10-Lite board was used. The board is shown in the figure below. It depicts the layout of the board and indicates the location of the connectors and key components.

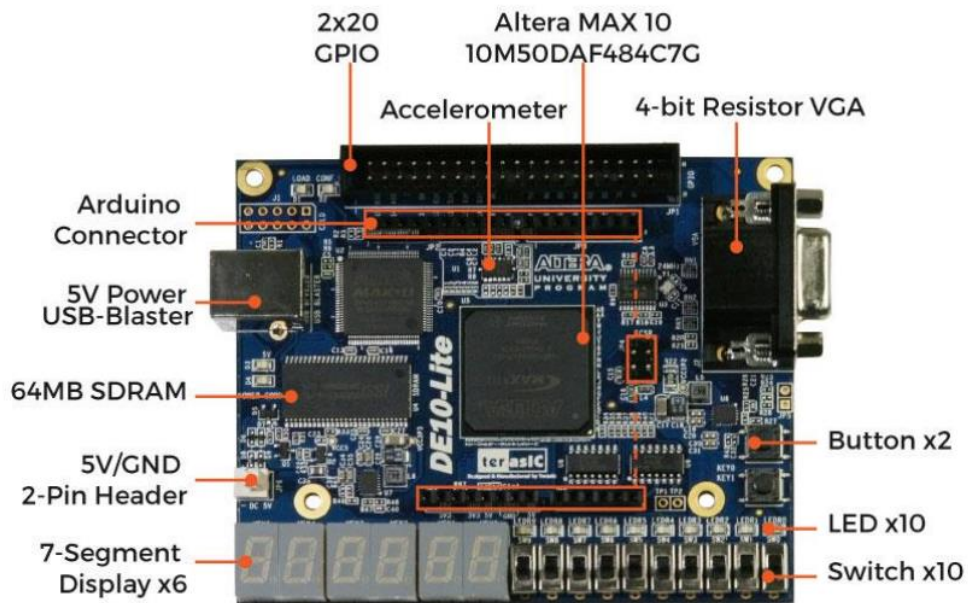


Figure 2: Connectors and key components DE10-Lite board.

## 2. Design and implementation

This chapter describes the implementation of the car parking system and the additional components with their code in VHDL. In the following paragraphs the implementation on the breadboard of the LEDs, resistors and buzzer is also shown.

The *DE10-Lite User Manual* [2] was consulted to connect the components to the FPGA.

### 2.1 Finite State Machine

The parking system has been implemented through a Mealy finite state machine which includes 4 states:

- State INITIAL,
- State PASSWORD,
- State OPEN,
- State CLOSE.

The following graph shows how the states interact with each other through the aid of ultrasonic sensors, whose implementation in VHDL is explained in the following paragraphs.

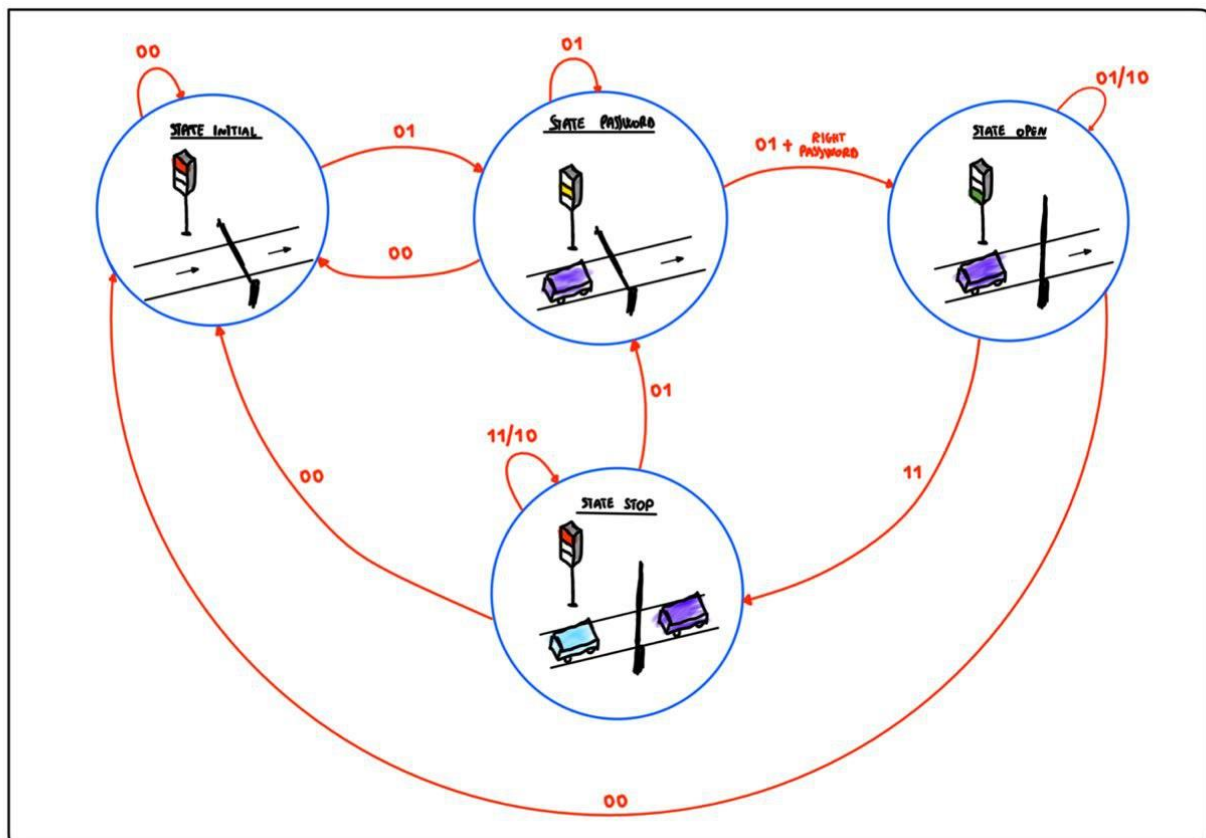


Figure 3: finite state machine graph.

The following VHDL code enforces a 1 second state transition. Also note the clock divider code: given the 50MHz internal clock, a 1Hz (i.e., 1s) clock was obtained using a counter.

```

TYPE state_values IS (initial, stpassword, stopen, stop);
SIGNAL pres_state, next_state: state_values;

SIGNAL counter: INTEGER := 1; -- Clock 1s
SIGNAL tmp: std_logic := '0';

-- CLOCK PER STATE MACHINE (1s)
BEGIN
    PROCESS (clk)
    BEGIN
        IF rising_edge(clk) THEN
            counter <= counter+1;
            IF counter = 25000000 THEN
                tmp <= not tmp;
                counter <= 0;
            END IF;
        END IF;
    END PROCESS;
    clk_out1 <= tmp;

    -- STATE MACHINE PER PARCHEGGIO
    statereg: PROCESS(tmp,reset)
    BEGIN
        IF (reset = '0') THEN
            pres_state <= initial;
        ELSIF rising_edge(tmp) THEN
            pres_state <= next_state;
        END IF;
    END PROCESS;

```

Figure 4: In order: signal definition, 1s clock divider, state machine initialization

The arcs in the graph indicate the value of the front sensor and the back sensor (*ECHO\_in* vector). Between the PASS and OPEN states, the arc as well as having the value "01" also requires that the password is correct. The latter has been implemented through a vector of 10 elements (figure 5) using FPGA switches. The password is unique ("0000000001").

```

-- VETTORE PASSWORD
password: IN std_logic_vector(9 downto 0);

```

Figure 5: implementation of the password vector in the entity.



Figure 6: FPGA switches and pin planner.

The implementation of states and interactions takes place with the following code.

```
fsm: PROCESS(pres_state, ECHO_in)
BEGIN
  CASE pres_state IS
    WHEN initial =>
      CASE ECHO_in IS
        WHEN "00" => next_state <= initial;
        WHEN "01" => next_state <= stpassword;
        WHEN "10" => next_state <= initial;
        WHEN "11" => next_state <= initial;
      END CASE;

    WHEN stpassword =>
      CASE ECHO_in IS
        WHEN "00" => next_state <= initial;
        WHEN "01" => CASE password IS
          WHEN "0000000001" => next_state <= stopen;
          WHEN OTHERS => next_state <= stpassword;
        END CASE;
        WHEN "10" => next_state <= initial;
        WHEN "11" => next_state <= stpassword;
      END CASE;

    WHEN stopen =>
      CASE ECHO_in IS
        WHEN "00" => next_state <= initial;
        WHEN "01" => next_state <= stopen;
        WHEN "10" => next_state <= stopen;
        WHEN "11" => next_state <= stop;
      END CASE;

    WHEN stop =>
      CASE ECHO_in IS
        WHEN "00" => next_state <= initial;
        WHEN "01" => next_state <= stpassword;
        WHEN "10" => next_state <= stop;
        WHEN "11" => next_state <= stop;
      END CASE;

    WHEN OTHERS => next_state <= initial;

  END CASE;
END PROCESS;
```

Figure 7: VHDL code for fsm implementation.

Initially, when both sensors do not detect any movement ("00" value), the system is in the INITIAL state and the following outputs are obtained:

- the 7-segment display shows "HI",
- the only led on is the red led,
- the gate is closed.

For the 7-segment display, reference was made to:

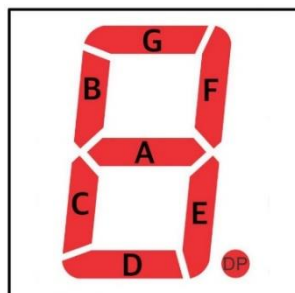


Figure 8: 7-segment display.



The VHDL code for the outputs is:

```
outputs: PROCESS (pres_state, ECHO_in, tmp, tmp1)
BEGIN
  CASE pres_state IS
    WHEN initial => ledr <= '1'; -- LED rosso acceso
                  ledg <= '0'; -- LED verde spento
                  ledy <= '0'; -- LED giallo spento

    -- BUZZER
    buzzer <= '0';

    -- SERVO MOTORE
    pwm <= segnaleServoZero ; -- Cancelli chiuso (0°)

    -- DISPLAY "HI"
    HEX1 <= "1001111"; -- I (1 a sinistra)
    HEX2 <= "0001001"; -- H
    HEX3 <= "1111111"; -- off
    HEX4 <= "1111111"; -- off
```

Figure 9: Initial state outputs code.

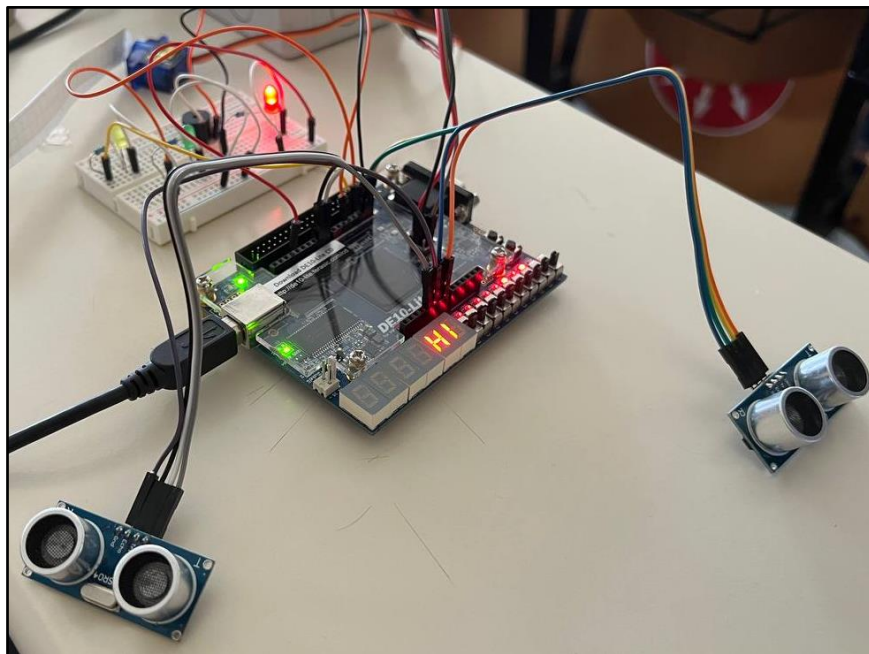


Figure 10: Initial state outputs

To go to the next state, it is necessary for the front sensor to detect the presence of an object (value "01") constantly, otherwise the system will return to the initial state. In no other case will it go from the initial state to the next one.

When the system is in the PASSWORD state, the following outputs occur:

- the 7-segment display shows "**PASS**",
- the red LED is on and the yellow LED flashes with a frequency of 1 second,
- the gate is closed.

The VHDL code for the outputs is:

```
WHEN stpassword => ledr <= '1';    -- LED rosso acceso
                  ledg <= '0';    -- LED verde spento

-- LED giallo lampeggiante
ledy <= tmp;

-- Buzzer
CASE password IS
WHEN "0000000000" => NULL;
WHEN "0000000001" => NULL;
WHEN others => buzzer <= tmp2;|
END CASE;

-- SERVO MOTORE
pwm <= segnaleServoZero ;    -- Cannello chiuso (0°)

-- DISPLAY "PASS"
HEX1 <= "0010010"; -- S (5)
HEX2 <= "0010010"; -- S (5)
HEX3 <= "0001000"; -- A
HEX4 <= "0001100"; -- P
```

Figure 11: Password state outputs code.

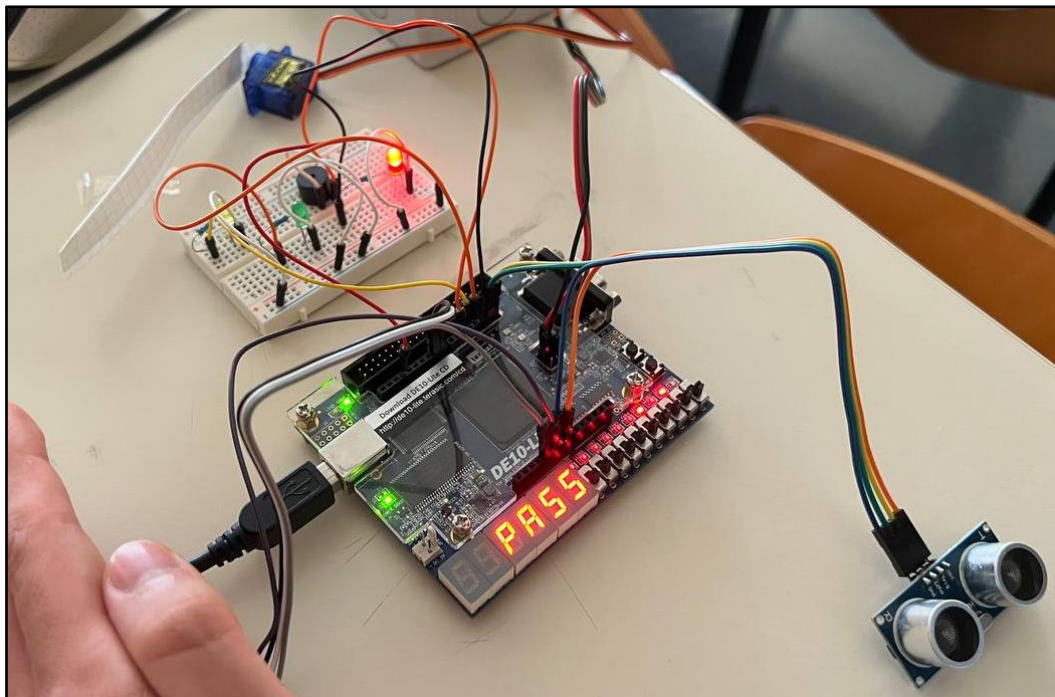


Figure 12: Password state outputs

In order to move to the next state, OPEN, the password entered must be correct. Otherwise, the passive buzzer will sound intermittently (the implementation will be explained in the following paragraphs). This check is done by the case-when statement.

When the system is in the OPEN state, the following outputs will occur:

- the 7-segment display shows "**GO IN**",
- the only LED on is the green LED,

- the gate is open.

The VHDL code for the outputs is:

```
WHEN stopen => ledg <= '1'; --LED verde acceso
               ledy <= '0'; --LED giallo spento
               ledr <= '0'; --LED rosso spento

               -- BUZZER
               buzzer <= '0';

               -- SERVO MOTORE
               pwm <= segnaleServoNovanta; -- Cannello aperto (~ 90°)

               -- DISPLAY "GO IN"
               HEX1 <= "1001000"; -- N
               HEX2 <= "1111001"; -- I (1 a destra)
               HEX3 <= "1000000"; -- 0 (0)
               HEX4 <= "0000010"; -- G (6)
```

Figure 13: Open state outputs code.

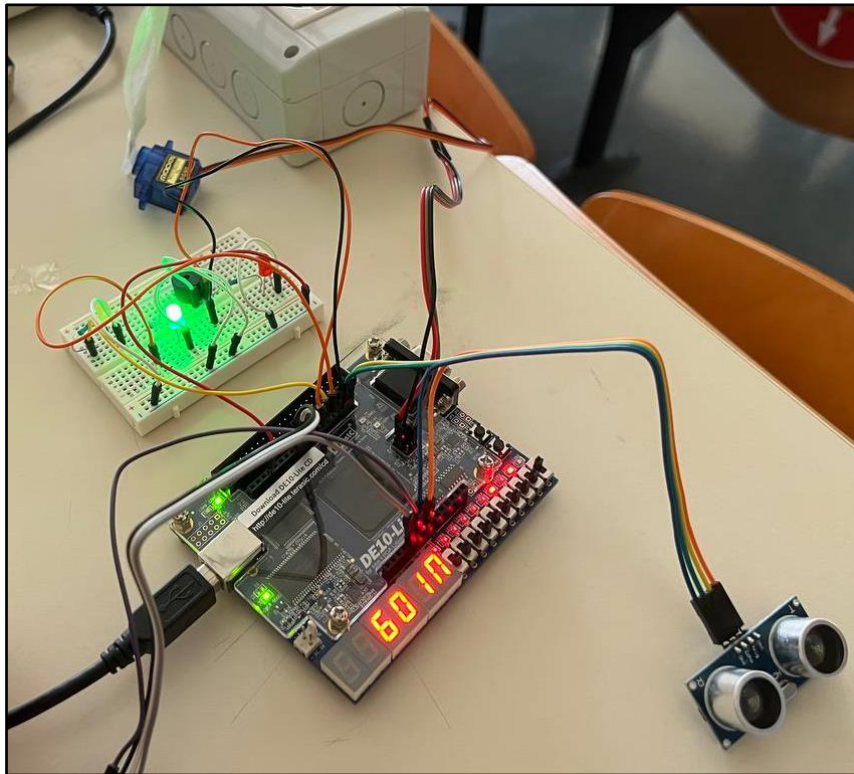


Figure 14: Open state outputs.

If the front sensor detects nothing, but the back sensor does (the car is entering the parking lot), the system remains in the OPEN state. If both sensors detect something, value "11", (there is a car waiting at the entrance and the previous car is still entering the parking lot), the system switches to the next state, STOP.

The outputs of this state are:

- the 7-segment display shows "**STOP**",
- the green LED is on and the red LED flashes with a frequency of 1 second,

- the gate is open.

The VHDL code for the outputs is:

```
    WHEN stop => ledg <= '1'; --LED verde acceso
                  ledy <= '0'; --LED giallo spento
                  ledr <= tmp; -- LED rosso lampeggiante

                  -- BUZZER
                  buzzer <= '0';

                  -- SERVO MOTORE
                  pwm <= segnaleServoNovanta; -- Cancelllo aperto (~ 90°)

                  -- DISPLAY "STOP"
                  HEX1 <= "0001100"; -- P
                  HEX2 <= "1000000"; -- 0 (0)
                  HEX3 <= "0000111"; -- t
                  HEX4 <= "0010010"; -- S (5)

    WHEN others => ledg <= '0';
END CASE;
END PROCESS;
```

Figure 15: Stop state outputs code.

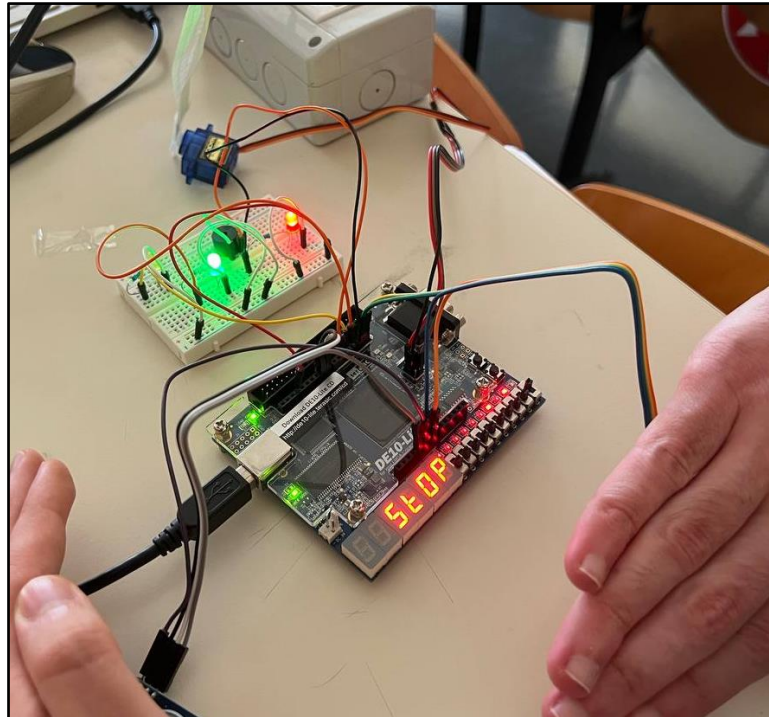


Figure 16: Stop state outputs.

Only once the previous car has entered the parking lot, so the back sensor detects nothing, the system switches to the PASSWORD state.

The system resets and returns to the INITIAL state only when both sensors detect nothing.

## 2.2 Breadboard implementation

A breadboard was used to physically implement the three LEDs, the three resistors and the passive buzzer. The LEDs used have a different supply voltage (red=1.8V, yellow=1.9V, green=2V) but with the 220 ohm resistors the operation of the LEDs is guaranteed.

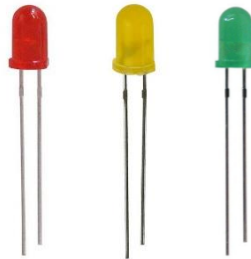


Figure 15: red, yellow, green LEDs.

To build the circuit, we connect one terminal of the resistor to one pin of the FPGA. We connect the longer pin of the LED (the positive, anode) to the other terminal of the resistor. Finally, we connect the short pin of the LED (the negative, cathode) to GND.

Buzzer pins are as follows:

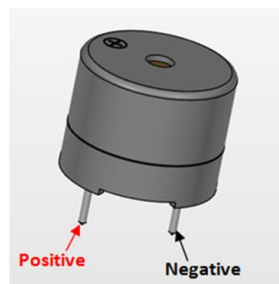


Figure 18: buzzer pin

- “Positive” is the signal to be supplied to the buzzer.
- “Negative” serves as a common ground (GND).

The breadboard has been connected through the GND pin of the FPGA which is common to the three circuits of the LEDs and the buzzer. However, each circuit is connected to a different digital pin, as shown in the pin planner below:

|     |      |        |         |     |        |        |         |
|-----|------|--------|---------|-----|--------|--------|---------|
| out | ledg | Output | PIN_AB8 | out | buzzer | Output | PIN_Y10 |
| out | ledr | Output | PIN_AB7 |     |        |        |         |
| out | ledy | Output | PIN_AB9 |     |        |        |         |

Figure 19: LEDs and buzzer pin planner.



For ease of visualization a sketch was made on Fritzing:

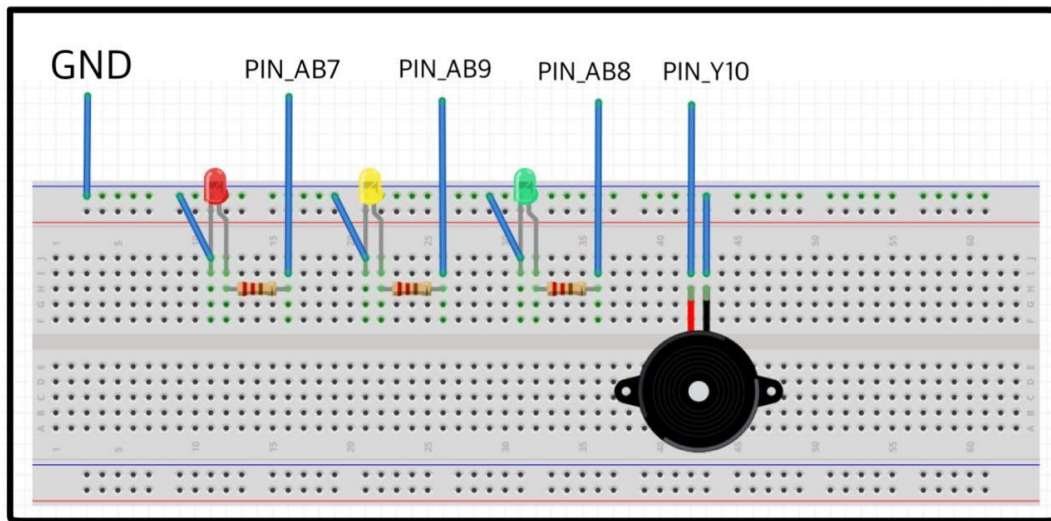


Figure 20: Fritzing sketch.

The actual implementation is as follows:

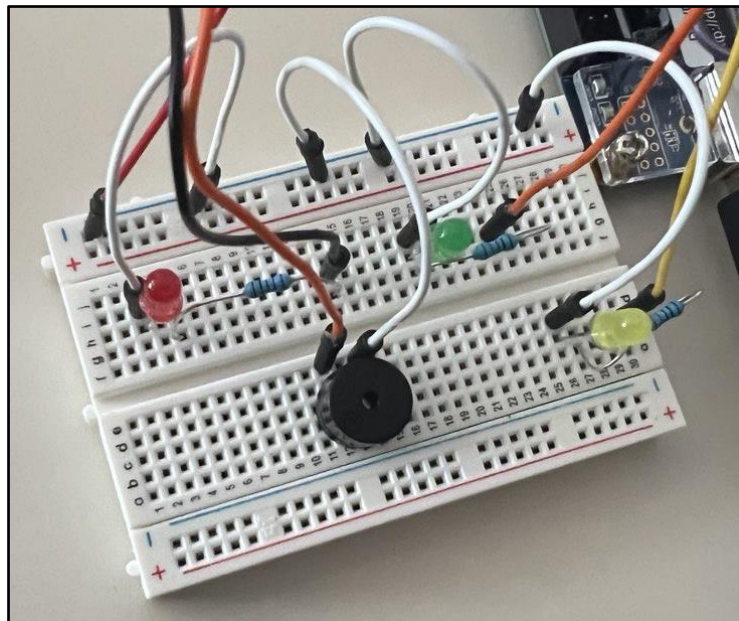


Figure 21: actual breadboard implementation.

## 2.3 Wrong Password Alert

The wrong password alert was implemented through a passive buzzer.



Figure 22: passive buzzer.

A passive buzzer was chosen because:

- Active buzzers have an internal oscillator, i.e. by powering them they begin to emit the typical sound. We therefore have no control over the oscillation frequency, and hence the tone of the sound.
- Passive buzzers do not have an internal oscillator, so they need an external power source to supply them with a square wave.

To simulate an incorrect password warning, the buzzer should be given an intermittent signal.

### 2.3.1 Buzzer Implementation

It is necessary to produce a signal with a high frequency so that the 'volume' of the sound is high but at the same time a low frequency so that it is audible to the human ear. In fact, it is necessary to consider that the range of human hearing is from around 20 hertz up to 20.000 hertz.

The idea is to generate a signal that has a high frequency in one half-period and a zero value in the other, thus, to generate an intermittent signal.

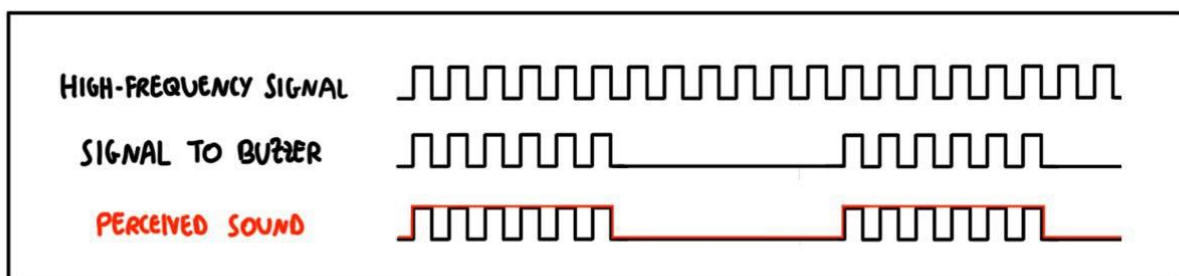


Figure 23: Buzzer implementation idea.

Considering the 50Mhz internal clock, a clock with a period of 0.5ms (i.e 2kHz) was created.

```
-- FREQUENZA PER BUZZER (2 kHz)
PROCESS (clk)
BEGIN
    IF rising_edge(clk) THEN
        counter1 <= counter1 + 1;
        IF counter1 = 12500 THEN    -- 0.5 ms = 2 kHz
            tmp1 <= not tmp1;
            counter1 <= 0;
        END IF;
    END IF;
END PROCESS;
clk_out3 <= tmp1;
```

Figure 24: high-frequency signal.

The signal to be supplied to the buzzer is generated by the 50 MHz internal clock.

We consider a counter, if it is less than or equal to 1s we impose that the signal sent to the buzzer (tmp2) is equal to the high frequency signal generated previously. If it is greater than 1s and less than 2s then we impose the signal to be zero and reset the counter.

```
-- CLOCK PER BUZZER
process(clk)
begin
    if rising_edge(clk) then
        if counter2 <= 25000000 then    -- 1 secondo
            counter2 <= counter2 + 1;
            tmp2 <= tmp1;

        elsif counter2 > 25000000 and counter2 < 50000000 then
            counter2 <= counter2 + 1;
            tmp2 <= '0';

        elsif counter2 = 50000000 then    -- 2 secondi
            counter2 <= 0;
        end if;
    end if;
end process;
```

Figure 25: generation of the signal to be supplied to the buzzer (tmp2).

The signals and outputs used are shown below.

```
SIGNAL counter1: INTEGER := 1; -- 0.5ms clock (high-frequency)
SIGNAL tmp1: std_logic := '0';

SIGNAL counter2: INTEGER := 1; -- by intermittence
SIGNAL tmp2: std_logic := '0';
```

Figure 26: buzzer clock signals.

```
clk_out3: OUT std_logic;
```

Figure 27: buzzer clock outputs.



Finally, in the output process of the state machine, we only impose the signal "tmp2" on the buzzer if the password is wrong. In other cases, we impose it to be zero.

```
buzzer: OUT std_logic;
```

*Figure 28: signal supplied to the buzzer.*

```
buzzer <= '0';  
buzzer <= tmp2;
```

*Figure 29: set buzzer.*

## 2.4 HC-SR04 ultrasonic sensor implementation

In this project to detect the presence of a vehicle in proximity of the gate we used two HC-SR04 ultrasonic distance sensors. The first one at the entrance the parking lot and the second on the other side of the gate to maintain it open when entering.

In general, an ultrasonic sensor works on the principle of SONAR and RADAR system which is used to measure the distance of an object.

The sensor generates a high frequency sound wave that is reflected as an echo when hitting an object and sensed by a receiver.

The distance is calculated by measuring the time required for the echo to reach the receiver.

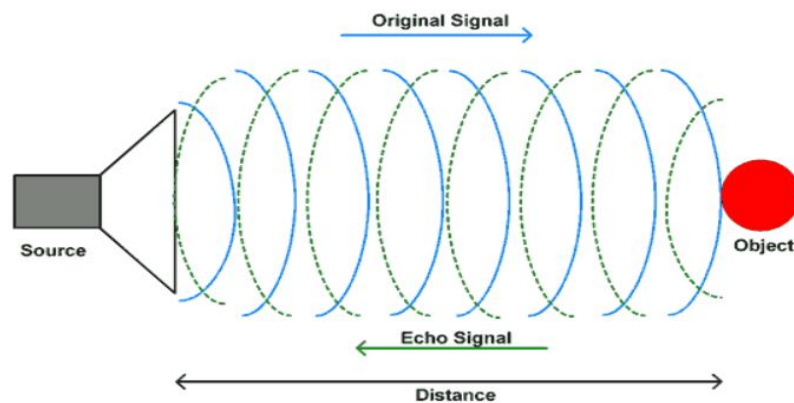


Figure 30: Ultrasonic sensor working principle.

The **HC-SR04** sensor consists in two transducers, one acts as an ultrasonic transmitter, the other as a receiver. The transmitter converts the electrical signal in a sound wave made by 8 pulses of 40 kHz and the receiver listens for the transmitted pulses.

Pins:

- **Vcc:** this pin supplies power to the HC-SR04 and is connected to the 5V output of the board.
- **Trigger:** when this pin is set to high for 10  $\mu$ s the sensor generates the pulses.
- **Echo:** this pin goes high when the sound wave is transmitted and remains high until the sensor receives an echo after which goes low.
- **Gnd:** this pin is connected to the ground of the board.



Figure 31: Ultrasonic module.

When the trig pin is set to high for at least 10  $\mu$ s the HC-SR04 sensor automatically transmits the ultrasonic burst of eight pulses at 40kHz (this kind of sound wave is designed so that the receiver can distinguish the transmitted pulses from ambient ultrasonic noise). The sensor waits for the rising edge at Echo pin.

When the rising edge occurs at Echo pin a timer start and the sensor waits for the falling edge at the Echo pin.

As soon as the Echo pin signal goes low the timer stops, this timer counts the time needed by the sensor to detect an object and return back.

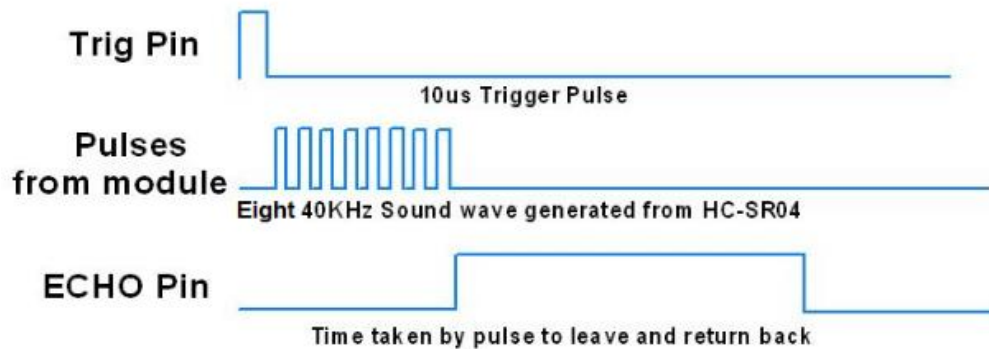


Figure 32: Ultrasonic HC-SR04 module timing diagram.

If the pulses transmitted are not reflected, the echo signal times out and goes low after 38ms, an echo signal of 38ms indicates no obstruction within the range of the sensor.

If those pulses are reflected, the echo pin goes low as soon as the signal is received. This generates a pulse on the echo pin that varies from 150  $\mu$ s to 25ms depending on the time taken to receive the signal.

To measure the distance:  $Distance = \frac{Speed\ of\ sound\ \left(343\frac{m}{s}\right) * Time\ when\ Echo\ is\ high}{2}$

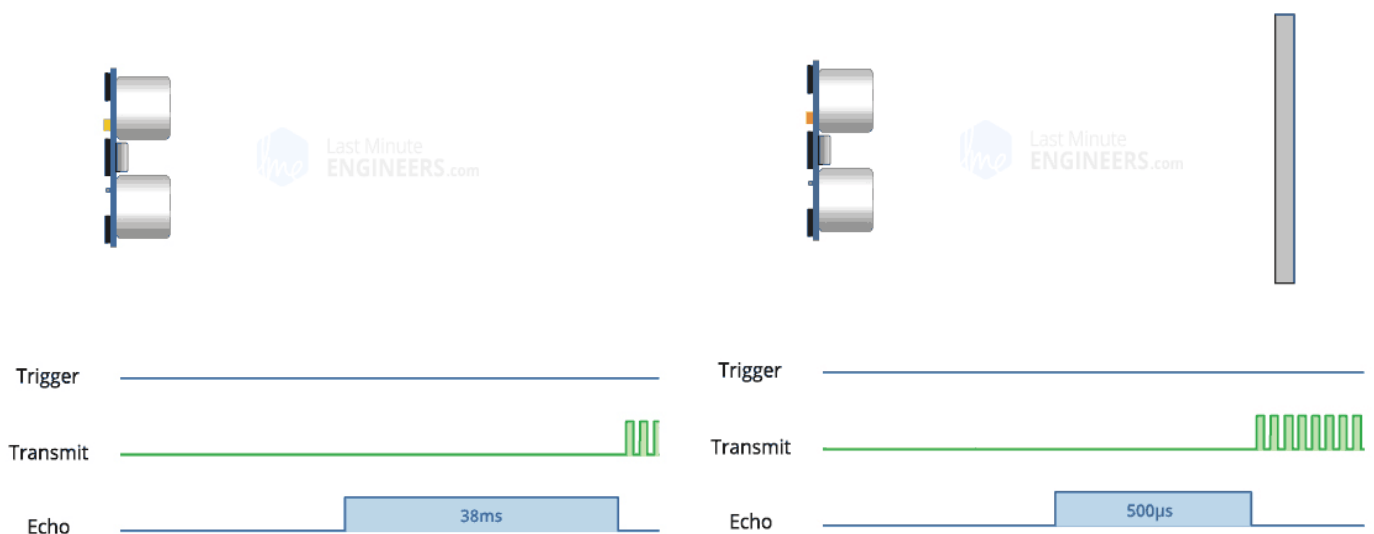


Figure 33: Ultrasonic sensor working principle representation with trigger and echo signal.

To implement the working principles of the sensors in the VHDL code we declared:

- “ECHO1” and “ECHO2” are the signal at the echo pin and are taken as input from the sensors.
- “trigger1” and “trigger2” are the trigger pulses that will be sent to the sensors.
- “count11” and “count22” (one for each sensor), two vectors that will be used to count the time when the signal at the echo pin is high. So, they will be needed to detect the presence of the vehicle at a certain distance from the sensors.
- “x” (for the front sensor) and “y” (for the back sensor) will have value ‘1’ if the vehicle is at a certain distance from the sensors, ‘0’ otherwise.
- “ECHO\_in” a two bits vector that will store x and y and will be used in the state machine.

```
-- ECHO DEI DUE SENSORI          -- TRIGGER PER I DUE SENSORI
ECHO1, ECHO2: IN std_logic;      trigger1, trigger2: OUT std_logic;

-- Segnali per Sensori
SIGNAL count11: std_logic_vector(17 downto 0); -- Counter per distanza
SIGNAL count22: std_logic_vector(17 downto 0);

SIGNAL ECHO_in: std_logic_vector(1 downto 0); -- Vettore Echo dei sensori
SIGNAL x,y: std_logic; -- x: Sensore di ingresso -- y: Sensore di uscita
```

Figure 34: variables and signals declared for the sensors.

Also, we wrote a code for a clock divider to realize a signal “tmp0” of 2μs period. This will be used to create the trigger pulse.

```
PROCESS (clk)
BEGIN
    IF rising_edge(clk) THEN
        counter0 <= counter0 + 1;
        IF counter0 = 25 THEN
            tmp0 <= not tmp0;
            counter0 <= 0;
        END IF;
    END IF;
END PROCESS;
clk_out2 <= tmp0;
```

Figure 35: clock divider to realize a 2 μs period signal.

In these two processes for each sensor, we define the trigger pulse of 10μs using a variable (define inside the process “count1” and “count2”) that increases every rising edge of “tmp0”, the trigger is set to ‘1’ at the start then when the count variable arrives to 10 the trigger is brought to zero. The two trigger signals are timed differently to avoid interference.

Also is started the counting of the time when the echo signal is high.

```

process (tmp0)
  variable count1: integer:= 0;
  begin
    if rising_edge(tmp0) then
      if count1 = 0 then
        count11 <= "000000000000000000";
        trigger1 <= '1';
      elsif count1 = 10 then
        trigger1 <= '0';
      end if;
      if ECHO1 = '1' then
        count11 <= count11 + 1;
      end if;
      if count1 = 249999 then
        count1 := 0;
      else
        count1 := count1 + 1;
      end if;
    end if;
  end process;

```

```

process (tmp0)
  variable count2: integer:= 0;
  begin
    if rising_edge(tmp0) then
      if count2 = 124999 then
        count22 <= "000000000000000000";
        trigger2 <= '1';
      elsif count2 = 125009 then
        trigger2 <= '0';
      end if;
      if ECHO2 = '1' then
        count22 <= count22 + 1;
      end if;
      if count2 = 249999 then
        count2 := 0;
      else
        count2 := count2 + 1;
      end if;
    end if;
  end process;

```

Figure 36: processes to define the trigger signal of the two sensors.

In this process when the ECHO signal goes low if “count11” or “count22” are below a certain value the signals x or y are brought to one.

By doing so we set a maximum distance that the object must have to the sensor to be considered “present”.

Then x and y are assigned to the ECHO\_in vector.

```

process (ECHO1,ECHO2)
  begin
    if falling_edge(ECHO1) then
      if count11 < 1000 then
        x <= '1';
      else
        x <= '0';
      end if;
    end if;
    if falling_edge(ECHO2) then
      if count22 < 1000 then
        y <= '1';
      else
        y <= '0';
      end if;
    end if;
  end process;

  ECHO_in(0) <= x;
  ECHO_in(1) <= y;

```

Figure 37: processes to create the ECHO\_in vector, which is used in the fsm.

The pins used for the two sensors are shown below:

|    |       |       |          |     |          |        |          |
|----|-------|-------|----------|-----|----------|--------|----------|
| in | ECHO1 | Input | PIN_AA17 | out | trigger1 | Output | PIN_AB17 |
| in | ECHO2 | Input | PIN_AB6  | out | trigger2 | Output | PIN_AB5  |

Figure 38: sensors pin planner.

The Vcc and GND pin of each sensor were connected to the board.

## 2.5 The Parking Gate

The parking gate was implemented through a servo motor, in particular, a SERVO MICRO TOWERPRO™ SG90 9G [5].



Figure 39: Servo Micro TOWERPRO™ SG90 9G.

A servo motor is a type of motor whose angle of rotation can be precisely controlled. When it receives a control signal, it then rotates to the required position.

### 2.5.1 Servo Motor Working Principle

The working principle of the Servo is very simple. Inside it includes a DC motor, a potentiometer and the control circuit.

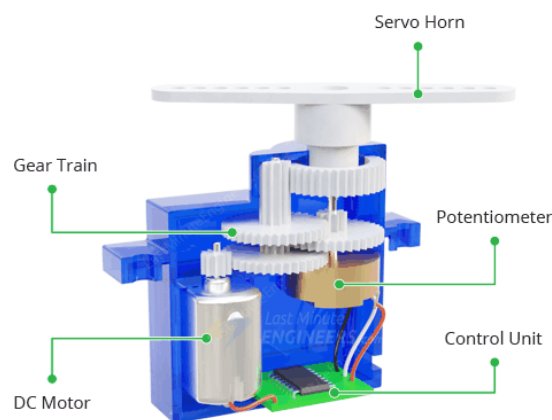


Figure 40: Internal components of a Servo

The *potentiometer* detects whether the shaft has moved to the required position by providing position feedback to the *control unit*, which compares the current motor position with the target position. When the motor shaft reaches the desired position, the power supplied to it will be removed.

The rotation speed will be based on the error between the required angle and the actual angle. If the motor is close to the required angle, the rotation speed is small, otherwise it is large.

This mechanism is known as a *servomechanism*. It is a closed-loop control system that uses negative feedback to adjust the motor's speed and direction to achieve the desired result.

The servo motor can be controlled by sending it a PWM signal. A typical servo motor expects a pulse every 20 milliseconds (50Hz). This means that the servo receives a new position command every 20ms and if we need the servo to hold position, we should continuously send the PWM signal to it.

The pulse width determines the position of the servo motor. They are proportional to each other.

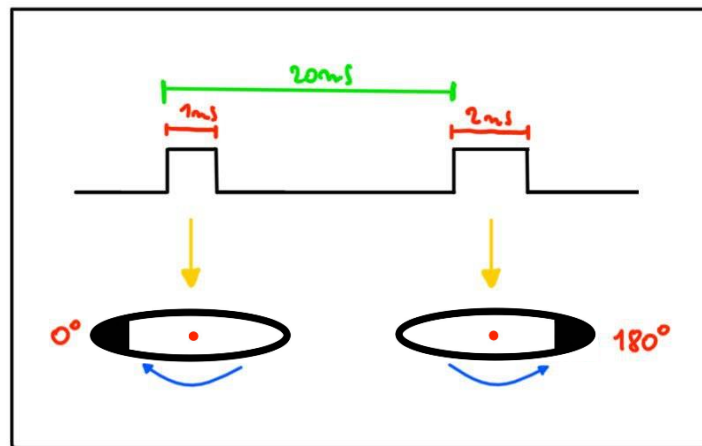


Figure 41: The pulse width in relation to the position of the servo motor.

Most Servos expect to see a pulse width between 1 and 2 ms:

- A pulse of 1ms or less will rotate the servo to 0 degrees.
- A pulse duration of 2ms will rotate the servo to 180 degrees.

The rotation angle '0-180 degrees' is proportional to the pulse width '1ms-2ms'.

### 2.5.2 Servo Motor Implementation

There are two parking gate configurations: closed or open.

When the gate is closed, the servo position must be 0 degrees; therefore, a PWM signal of 20ms period characterised by a 1ms pulse must be supplied to the servo motor.

When the gate is open, the servo position must be 90 degrees; therefore, a PWM signal of 20ms period characterised by a 1,5ms pulse must be supplied to the servo motor.



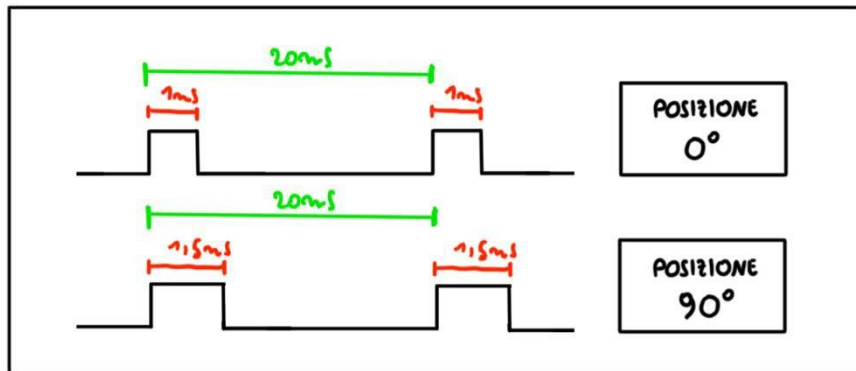


Figure 42: servo motor implementation idea.

Two clocks were created considering the internal clock of 50Mhz, in particular a 2ms period clock to generate a 1ms pulse and a 3ms period clock to generate a 1,5ms pulse.

```
-- SERVO MOTOR CLOCK
process(clk)
begin
    if rising_edge(clk) then
        countservo1 <= countservo1 + 1;
        if countservo1 = 49999 THEN
            tmpservo1 <= not tmpservo1;
            countservo1 <= 0;
        END IF;
    END IF;
END PROCESS;
clk_servo1 <= tmpservo1;

process(clk)
begin
    if rising_edge(clk) then
        countservo2 <= countservo2 + 1;
        if countservo2 = 74999 THEN
            tmpservo2 <= not tmpservo2;
            countservo2 <= 0;
        END IF;
    END IF;
END PROCESS;
clk_servo2 <= tmpservo2;
```

Figure 43: the two initial clocks to generate the two PWM signals.

The signals and outputs used are shown below.

```
-- Servo Motor Signals
signal countservo1: integer:=1;
signal countservo2: integer:=1;
signal tmpservo1: std_logic := '0';
signal tmpservo2: std_logic := '0';

clk_servo1, clk_servo2: out std_logic;
```

Figure 44: servo motor clock signals.

Figure 45: servo motor clock outputs.

Afterwards, these clocks were used to derive the desired PWM signals.

The logic for generating PWM signals is as follows: using a counter we consider a period of 20ms, the first step (counter = 1) we assign the PWM signal a high value and for the remaining steps (until 20ms is reached) a low value. Then the counter resets to zero and the cycle starts again.

```
-- To produce a signal that generates a 1ms pulse every 20ms
process (tmpservo1)
  variable counts1: integer := 0;
  begin
    if rising_edge(tmpservo1) then
      counts1 := counts1 + 1;
      if counts1 = 1 then --2ms (1ms pulse)
        segnaleServoZero <= '1';
      elsif counts1 > 1 then
        segnaleServoZero <= '0';
      end if;

      if counts1 = 10 then -- 20ms
        counts1 := 0;
      end if;
    end if;
  end process;

-- To produce a signal that generates a 1,5ms pulse every 20ms
process (tmpservo2)
  variable counts2: integer := 0;
  begin
    if rising_edge(tmpservo2) then
      counts2 := counts2 + 1;
      if counts2 = 1 then --3ms (1,5ms pulse)
        segnaleServoNovanta <= '1';
      elsif counts2 > 1 then
        segnaleServoNovanta <= '0';
      end if;

      if counts2 > 7 then -- ≈20ms
        counts2 := 0;
      end if;
    end if;
  end process;
```

Figure 46: PWM signals to be supplied to the servo motor.

The logic for generating the PWM signal to keep the servo at 0 degrees is shown below.

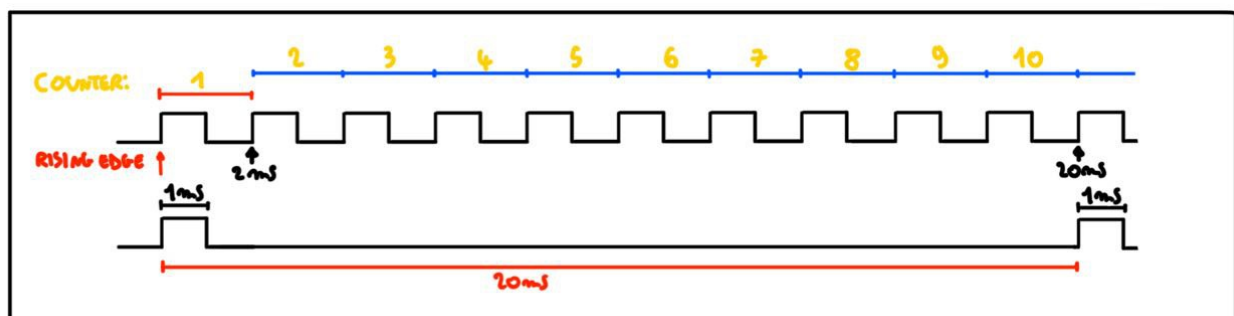


Figure 47: logic for generating the PWM signal to keep the servo at 0 degrees.

Finally, in the output process of the state machine, the PWM signals “SegnaleServoZero” and “SegnaleServoNovanta” were set depending on whether the gate is closed or open.

```
pwm : out std_logic;
```

Figure 48: signal supplied to the servo motor.

```
pwm <= segnaleServoZero ; -- Gate closed (0°)
```

```
pwm <= segnaleServoNovanta; -- gate open (90°)
```

Figure 49: set servo position

### 2.5.3 FPGA Connection

Servo motor pins are as follows:

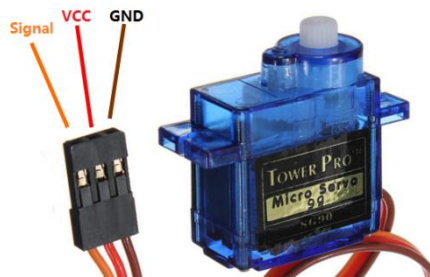


Figure 50: servo motor pin.

- “Signal” is an input for the control system (the PWM signal).
- “VCC” is a positive voltage that powers the servo (5V).
- “GND” serves as a common ground for the motor and the logic.

The chosen pins are the following:

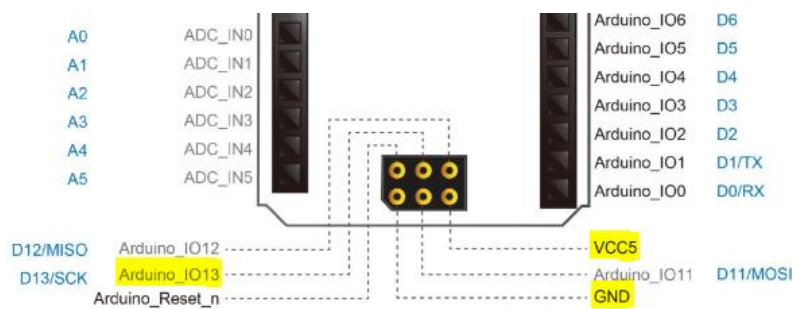


Figure 51: FPGA Pins chosen by DE10-Lite User Manual.

|              |          |              |     |              |
|--------------|----------|--------------|-----|--------------|
| Arduino_IO13 | PIN_AB20 | Arduino IO13 | SCK | 3.3-V LVTTTL |
|--------------|----------|--------------|-----|--------------|

Figure 52: FPGA pin name for PWM signal by DE10-Lite User Manual.

| Node Name                                                                             | Direction | Location |
|---------------------------------------------------------------------------------------|-----------|----------|
|  pwm | Output    | PIN_AB20 |

Figure 53: Pin Planner - PWM signal.

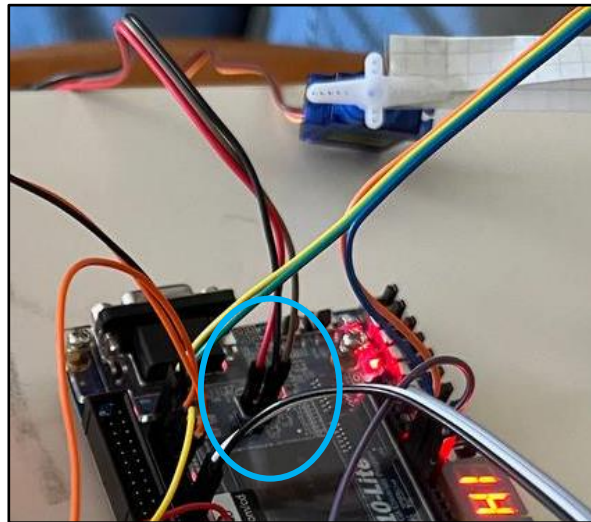


Figure 54: servo motor and FPGA connection

### 3. Conclusions and future implementations

The parking system is fully functional, however the way to enter the password is somewhat cumbersome. In fact, once the password has been entered through the switches, it must be reset manually before the arrival of a new user.

Another problem, as said in the paragraph about the implementation of the sensors, is that it was necessary to delay the trigger signal of one sensor with respect to the other to avoid interference. In fact, during the testing of the code, the generation of the trigger signals at the same time caused one sensor to output a wrong ECHO signal causing the state to change constantly.

In reference to the servo motor implementation, when the operation of the servo motor was tested, it was noticed that the servo did not position exactly at 0 degrees (something more) and 90 degrees (something less). One should therefore decrease the pulse width and increase it respectively. However, it was decided to leave the initial values set.

In conclusion, the project presents most of the implementations that the team had set. Future implementations may be:

- A keypad (4x4) for entering the password.



*Figure 55: 4x4 keypad.*

- A sound in the OPEN status to indicate that the entered password is correct, using the buzzer.
- A camera that reads the license plate of the car, and each license plate is associated with a different password [6].

## 4. References

- [1] "FPGA applications" - <https://hardwarebee.com/fpga-common-applications/>
- [2] "DE10-Lite User Manual" - <https://usermanual.wiki/Document/DE10LiteUserManual.444138977/html>
- [3] "HC-SR04 Ultrasonic Sensor Guide with Arduino Interfacing" - [HC-SR04 Ultrasonic Sensor Guide with Arduino Interfacing | Sensor... \(electronicwings.com\)](#).
- [4] "How HC-SR04 Ultrasonic Sensor Works & Interface It With Arduino" - [How HC-SR04 Ultrasonic Sensor Works & How to Interface It With Arduino \(lastminuteengineers.com\)](#).
- [5] "SERVO MICRO TOWERPRO SG90 9G" - <https://www.robotstore.it/Servo-micro-TowerPro-SG90-9g>
- [6] "A high speed license plate recognition system on an FPGA" – DOI: [10.1109/FPL.2007.4380715](#), [https://www.researchgate.net/publication/4290240\\_A\\_High\\_Speed\\_License\\_Plate\\_Recognition\\_System\\_on\\_an\\_FPGA](https://www.researchgate.net/publication/4290240_A_High_Speed_License_Plate_Recognition_System_on_an_FPGA)